

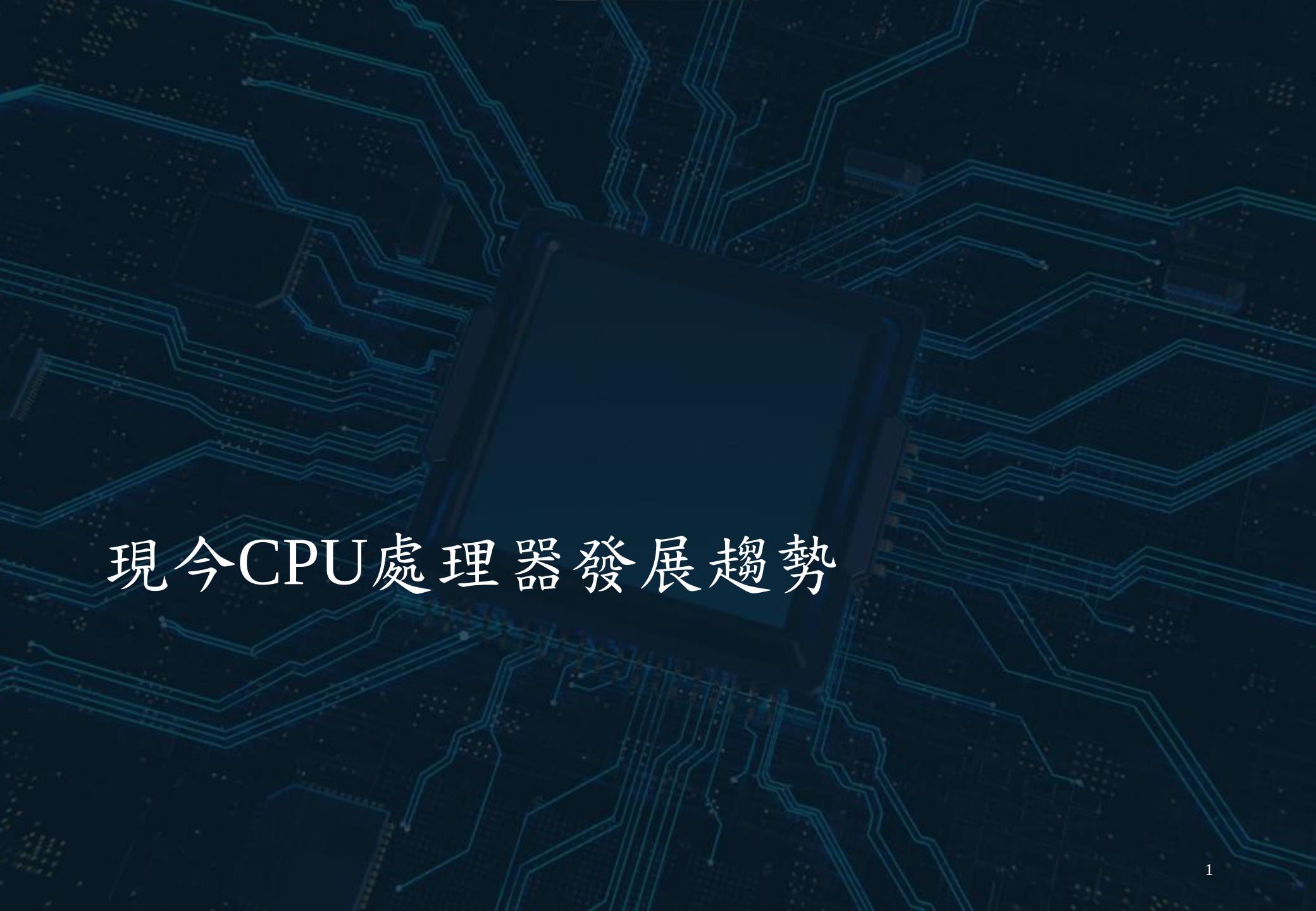
微算機系統 2026 MICROPROCESSOR SYSTEMS

Instructor : Yen-Lin Chen(陳彥霖)

Dept. Computer Science and Information Engineering

National Taipei University of Technology

ylchen@ntut.edu.tw



現今CPU處理器發展趨勢



CISC與RISC指令集

RISC vs. CISC

CISC	RISC
Emphasis on hardware	Emphasis on software
Multiple instruction sizes and formats	Instructions of same set with few formats
Less registers	Uses more registers
More addressing modes	Fewer addressing modes
Extensive use of microprogramming	Complexity in compiler
Instructions take a varying amount of cycle time	Instructions take one cycle time
Pipelining is difficult	Pipelining is easy

CISC代表：

Intel, AMD

RISC代表：

RISC-V, ARM, MIPS



AMD陣營CPU發展趨勢

微架構	推出年代	製程		備註
Zen	2017	14nm		
Zen+	2018	12nm		
Zen 2	2019	7nm	Ryzen 5 3600 Ryzen 9 3900X	(AMD真香)
Zen 3	2020	7nm	Ryzen 7 4800H Ryzen 9 5900X	(Mobile) (Desktop)
Zen 4	2022	5nm	Ryzen 5 7600 Ryzen 9 7950X3D	
Zen 4 3D V-Cache	2023	5nm	Ryzen 7 7800X3D	5nm + 3D堆疊快取
Zen 5 3D V-Cache	2024	4nm	Ryzen 9000X3D 系列	(Mobile 7月) (Desktop 8月)
Zen6	2026	3nm / 2nm		(Server 2026) (Desktop 2027)



Intel 陣營CPU發展趨勢

微架構	推出年代	製程	代表CPU	備註
Kaby Lake	2018	14nm+	i7-7700	
Coffee Lake	2017	14nm++	i7-8086K	x86 40周年
Coffee Lake Refresh	2019	14nm+++	i9-9900K	Core i9問世
Ice Lake	2019	10nm	i5-1035G7	(移動平台)
Comet Lake	2020	14nm++++	i7-10700	
Rocket Lake	2021	14nm+++++	i7-11700	
Alder Lake	2022	Intel 7	i7-12700	Hybrid 架構 (大小核)
Raptor Lake	Q1'2023	Intel 7	i7-13700	
Raptor Lake	Q4'2023	Intel 7	i7-14700	
Raptor Lake Refresh	2023	Intel 7	i9-14900K	改良版，提高時脈和快取
Meteor Lake	2024	Intel 4	Core Ultra 100 系列	首次導入 Tile 小晶片架構 與首代 NPU
Lunar Lake	2024	TSMC N3B	Core Ultra 200V 系列	記憶體直接封裝於處理器 晶片上 (MoP)



Intel 陣營CPU發展趨勢

微架構	推出年代	製程	代表CPU	備註
Arrow Lake	Q4'2024	TSMC N3B	Core Ultra 200S 系列	
Panther Lake	2026	Intel 18A	Core Ultra 300 系列	
Nova Lake	Q4'2026	Intel 18A	Core Ultra 400 系列	



ARM發展趨勢

ARM架構，過去稱作進階精簡指令集機器（Advanced RISC Machine），是一個精簡指令集（RISC）處理器架構家族，ARM處理器非常適用於行動通訊領域，符合其主要設計目標為低成本、高效能、低功耗的特性。

ARM公司本身並不靠自有的設計來製造或販售CPU，而是將處理器架構授權給有興趣的廠家。ARM提供了多樣的授權條款，也提供了ARM核心的整合硬體敘述，包含完整的軟體開發工具（編譯器、debugger、SDK），使這些廠家能根據自己的需求使用ARM提供的工具自行開發CPU。

指令集架構	處理器家族
ARMv1	ARM1
ARMv2	ARM2 、 ARM3
ARMv3	ARM6、 ARM7
ARMv4	StrongARM 、 ARM7TDMI 、 ARM9TDMI
ARMv5	ARM7EJ 、 ARM9E 、 ARM10E 、 XScale
ARMv6	ARM11 、 ARM Cortex-M
ARMv7	ARM Cortex-A 、 ARM Cortex-M 、 ARM Cortex-R
ARMv8	Cortex-A35、Cortex-A50系列 [18] 、Cortex-A70系列、Cortex-X1
ARMv9	Cortex-A510 、 Cortex-A710 、 Cortex-A715 、 Cortex-X2 、 Cortex-X3 、 ARM Neoverse N2

ARM CPU發展趨勢

高通

Snapdragon 8 Gen2 (Q2'2023)

- 1x Arm Cortex-X3 at 3.2GHz
- 2x Arm Cortex-A715 up to 2.8GHz
- 2x Arm Cortex-A710 up to 2.8GHz
- 3x Arm Cortex-A510v2 up to 2.0GHz

Snapdragon 8 Gen3 (Q4'2023)

- 1x Arm Cortex-X4 at 3.3GHz
- 3x Arm Cortex-A720 up to 3.15GHz
- 2x Arm Cortex-A720 up to 2.96GHz
- 2x Arm Cortex-A520 up to 2.27GHz

Snapdragon 8 Elite (Q4'2024)

- 1x Qualcomm Oryon (Prime) up to 4.32 GHz
- 6x Qualcomm Oryon (Performance) up to 3.53 GHz

Snapdragon 8 Elite Gen 5 (Q4'2025)

- 2x Qualcomm Oryon (Prime) up to 4.61 GHz
- 6x Qualcomm Oryon (Performance) up to 3.63 GHz

聯發科

MediaTek Dimensity 9200 (Q2'2023)

- 1x Arm Cortex-X3 at 3.05GHz
- 3x Arm Cortex-A715 up to 2.85GHz
- 4x Arm Cortex-A510v2 up to 1.8GHz

MediaTek Dimensity 9300 (Q4'2023)

- 4x Arm Cortex-X4 at 3.25GHz
- 4x Arm Cortex-A720 up to 2.0GHz

MediaTek Dimensity 9400 (Q4'2024)

- 1x Arm Cortex-X925 at 3.62 GHz
- 3x Arm Cortex-X4 at 3.30 GHz
- 4x Arm Cortex-A720 at 2.40 GHz

MediaTek Dimensity 9500 (Q3'2025)

- 1x Arm C1-Ultra at 4.21 GHz
- 3x Arm C1-Premium at 3.50 GHz
- 4x Arm C1-Pro at 2.70 GHz



ARM CPU發展趨勢

蘋果	CPU	GPU	Memory Bus Bit
M2	4P+4E	10Core	
M3	4P+4E	10Core RT	
M4	4P+6E	10Core RT	
M5	4P+6E	10Core RT	
M2 Pro	8P+4E	19Core	256bit
M3 Pro	6P+6E	18Core RT	192bit
M4 Pro	10P+4E	20Core RT	
M4 Pro	6S+12P	20Core RT	
M2 Max	8P+4E	38Core	
M3 Max	12P+4E	40Core RT	
M4 Max	12P+4E	40Core RT	
M5 Max	6S+12P	40Core RT	

高通

Snapdragon X Elite (Q4'2023)

- 12x 高通自研 Oryon at 3.8GHz (All big core)
- (其中有兩顆核心 up to 4.3GHz)
- 1x hexagon NPU (AI運算)

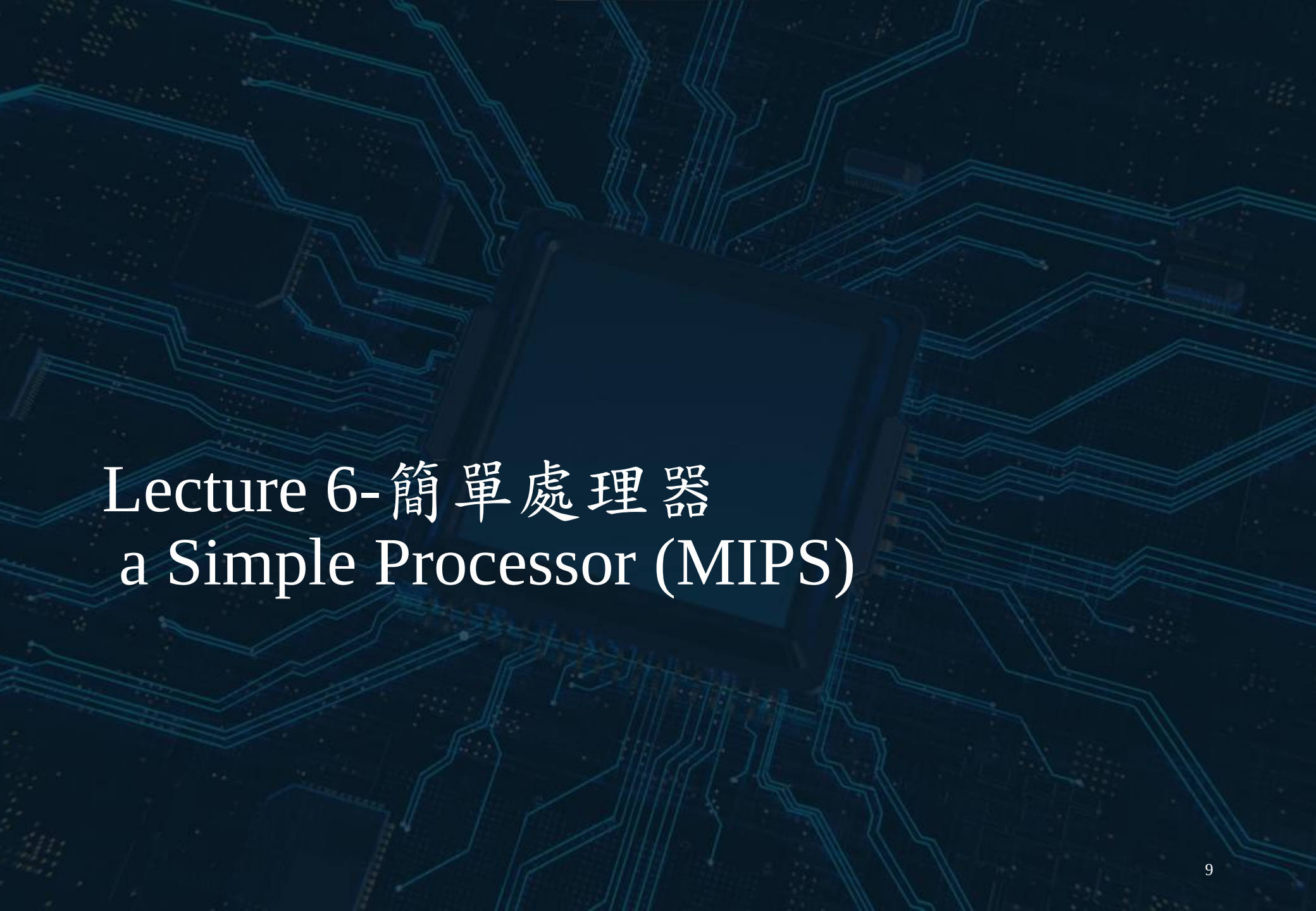
Snapdragon X2 Elite / Extreme(Q4'2025)

- 12x 高通自研 Oryon at 4.0GHz (混合核心架構)
- (其中有 兩顆核心 up to 4.5GHz)
- 1x hexagon NPU (AI運算)

Snapdragon X2 Elite / Extreme(Q4'2025)

- 18x 高通自研 Oryon at 4.2GHz (混合核心架構)
- (其中有 兩顆核心 up to 5.0GHz)1
- 1x hexagon NPU (AI運算)

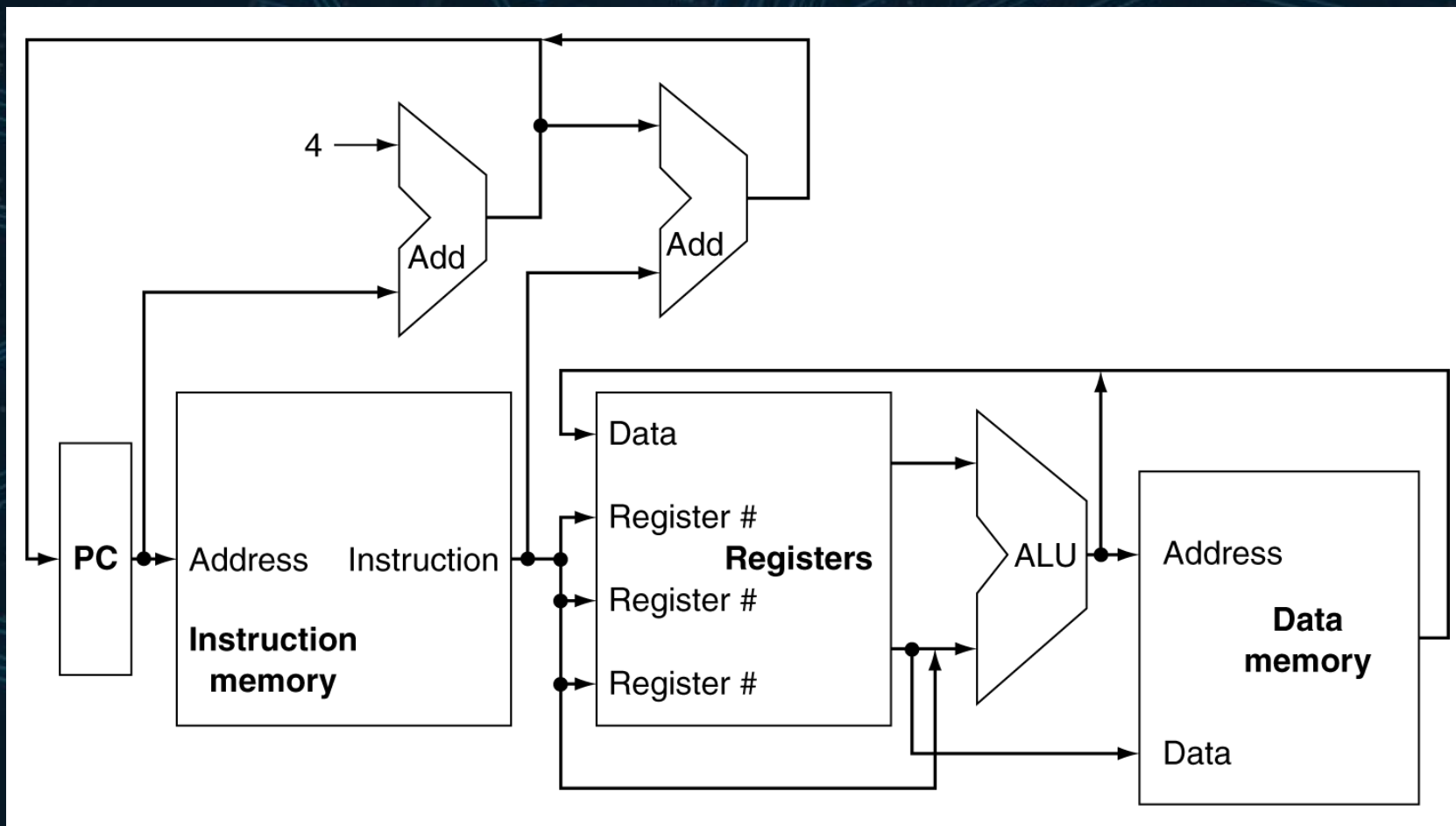
RT : Ray Tracing光追



Lecture 6-簡單處理器 a Simple Processor (MIPS)



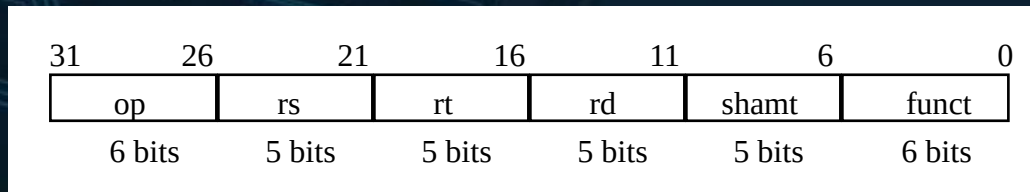
CPU 基本架構





Example: Analyze MIPS Instruction Set

- R-type MIPS instructions are 32 bits long:
 - R-type:



- These fields are:
 - op: operation of the instruction
 - rs, rt, rd: source and destination register
 - shamt: shift amount
 - funct: selects variant of the “op” field



Example: R-type MIPS Subset

R-Type:

add rd, rs, rt
sub rd, rs, rt
and rd, rs, rt
or rd, rs, rt
slt rd, rs, rt

31	26	21	16	11	6	0
op	rs	rt	rd	shamt	funct	
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	



一、簡單處理器 Simple Processor

- 執行表1運算類型的數位系統通常稱為處理器(processor)

運算	執行的功能
Load Rx, Data	$Rx \leftarrow Data$
Move Rx, Ry	$Rx \leftarrow [Ry]$
Add Rx, Ry	$Rx \leftarrow [Rx] + [Ry]$
Sub Rx, Ry	$Rx \leftarrow [Rx] - [Ry]$

表1 處理器執行的運算。

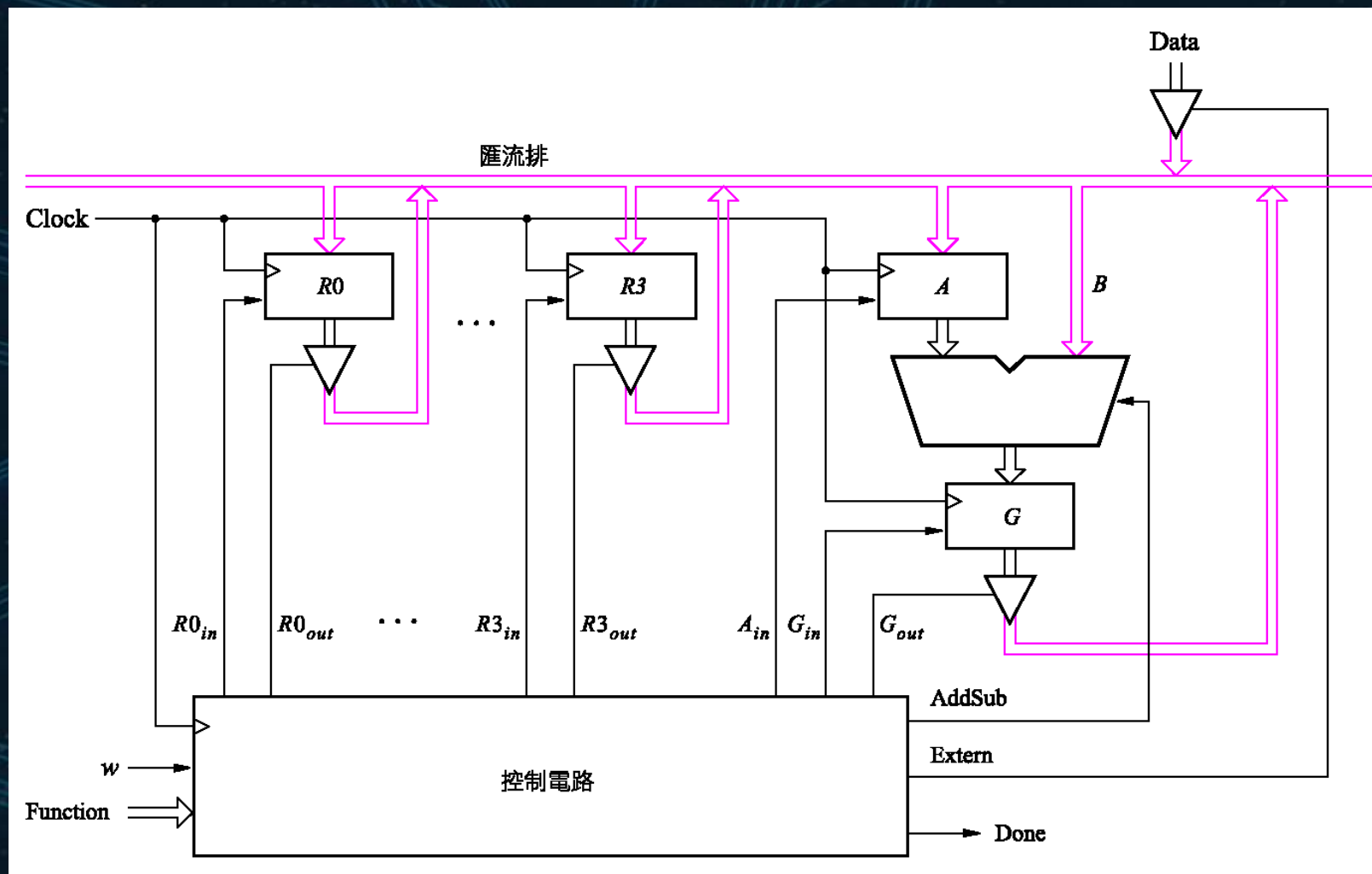


圖1 實作簡單處理器的數位系統。



一、簡單處理器 Simple Processor

- FR_{in} 訊號是用來將Function(指令)輸入(Fetch)的值載入5指令功能暫存器，當 w 變成1時。因此 $FR_{in} = wT_0$
- 而對於表示清除與重置處理器狀態其正確的邏輯表示式為 $Clear = \bar{w}T_0 + Done$ (Done表示指令完成)

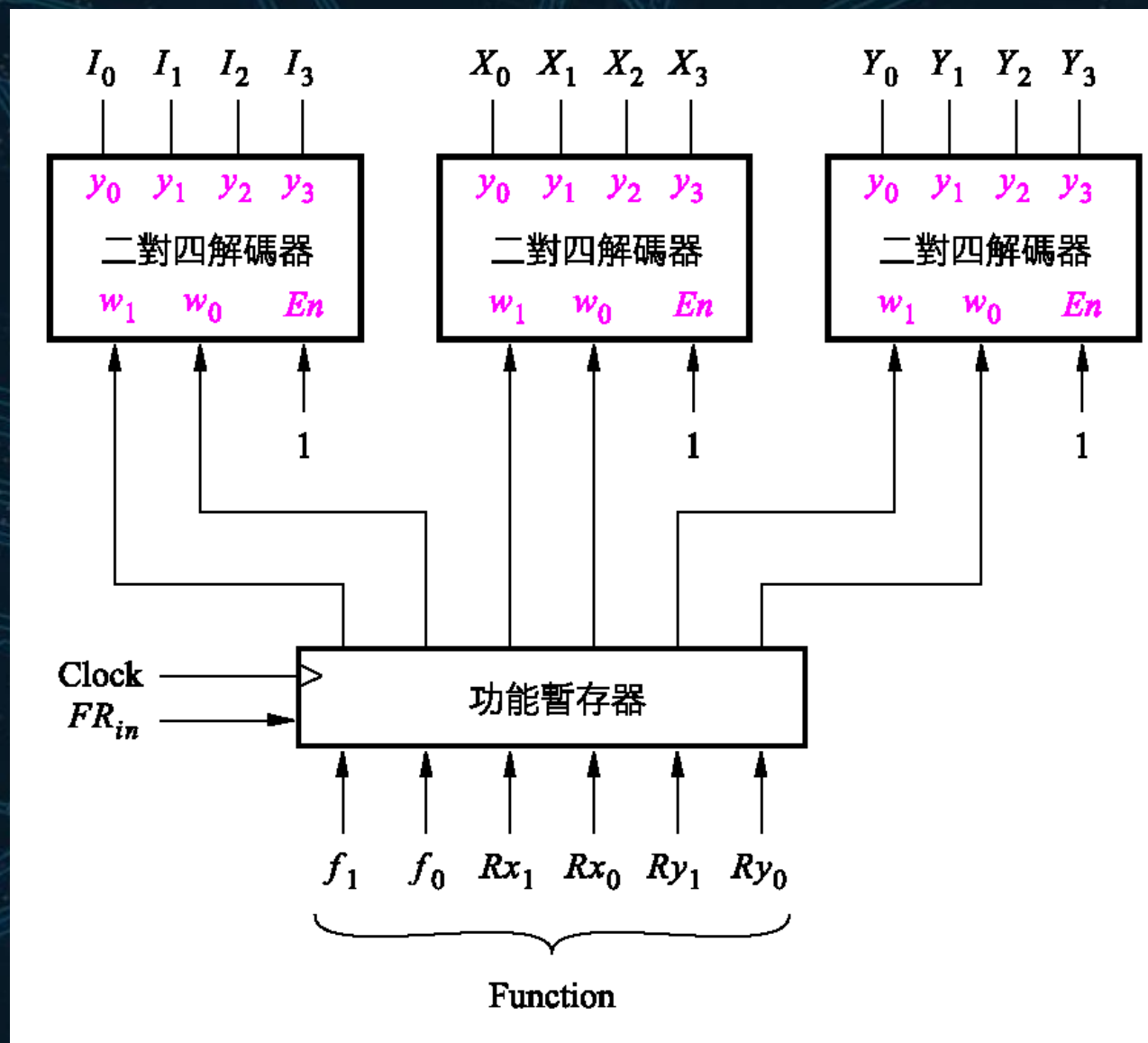


圖2 功能暫存器和解碼器。

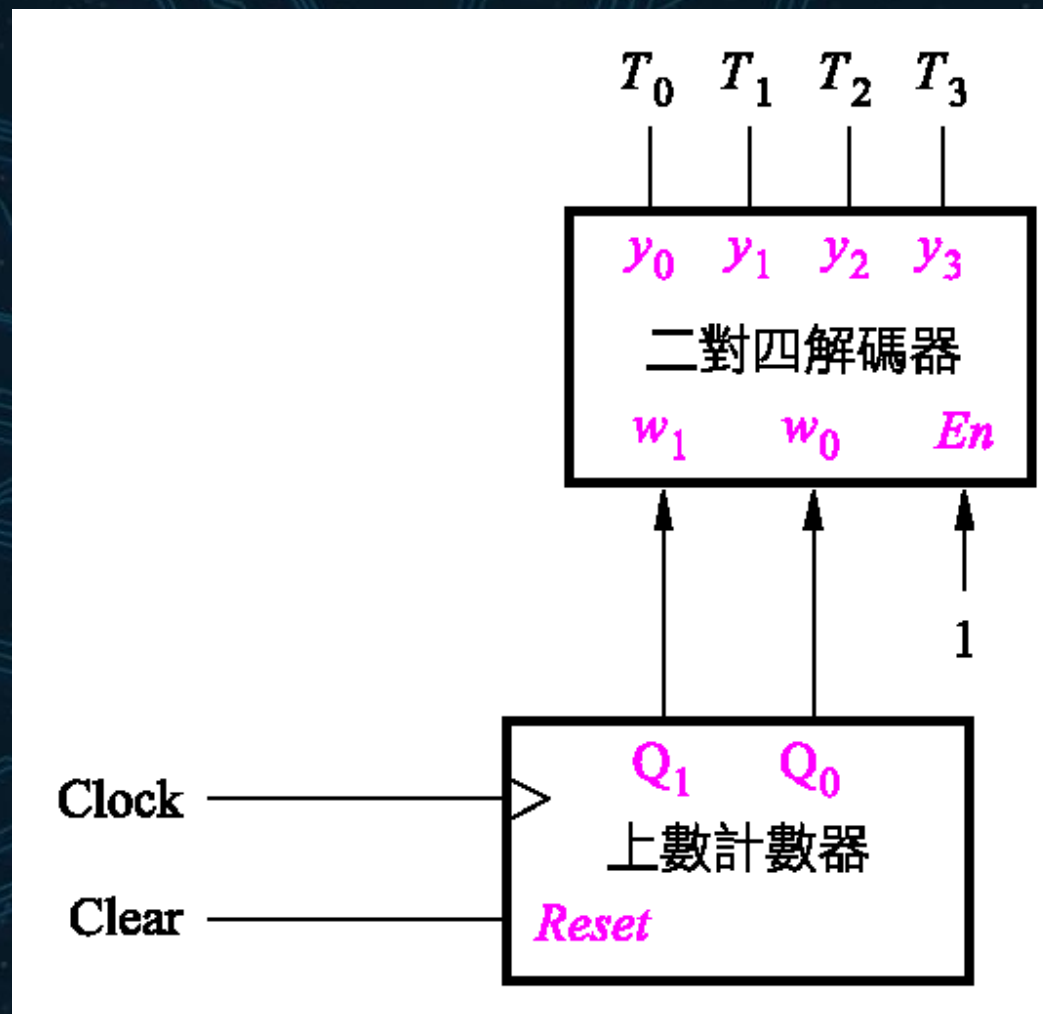


圖3 處理器控制電路的一部份。



一、簡單處理器 Simple Processor

- 控制電路的其他輸出跟每個運算執行的特定步驟有關。每個訊號必須產生的值如表2所示。

	T_1	T_2	T_3
(Load): I_0	$Extern, R_{in} = X$ $Done$		
(Move): I_1	$R_{in} = X, R_{out} = Y$ $Done$		
(Add): I_2	$R_{out} = X, A_{in}$	$R_{out} = Y, G_{in},$ $AddSub = 0$	$G_{out}, R_{in} = X,$ $Done$
(Sub): I_3	$R_{out} = X, A_{in}$	$R_{out} = Y, G_{in},$ $AddSub = 1$	$G_{out}, R_{in} = X,$ $Done$

表2 每個運算／時間步驟設定的控制訊號。



- 此表顯示， $R0_{in}$ 在Load和Move的第一步和Add和Sub的第三步被設為 X_0 值，產生下列表示式

$$R0_{in} = (I_0 + I_1)T_1X_0 + (I_2 + I_3)T_3X_0$$

- 類似地， $R0_{out}$ 在Move的第一步被設成 Y_0 值。它在Add和Sub的第一步被設為 X_0 值，在其他運算的第二步被設成 Y_0 值，產生

$$R0_{out} = I_1T_1Y_0 + (I_2 + I_3)(T_1X_0 + T_2Y_0)$$

- $R1_{in}$ 和 $R1_{out}$ 的式子和 $R0_{in}$ 和 $R0_{out}$ 一樣，只是用 X_1 和 Y_1 來取代 X_0 和 Y_0 。
 $R2_{in}$ 、 $R2_{out}$ 、 $R3_{in}$ 和 $R3_{out}$ 都可以相同方式得到。



- 因此，實作此訊號的邏輯表示式為

$$Extern = I_0 T_1$$

- 在Load和Move的第一步Done被設定，在Add和Sub的第三步也是。
因此

$$Done = (I_0 + I_1)T_1 + (I_2 + I_3)T_3$$

- A_{in} 、 G_{in} 和 G_{out} 訊號在Add和Sub運算被設定。
- 此由下面邏輯表示式達成

$$A_{in} = (I_2 + I_3)T_1$$

$$G_{in} = (I_2 + I_3)T_2$$

$$G_{out} = (I_2 + I_3)T_3$$

$$AddSub = I_3$$



二、VHDL程式碼 VHDL Code

- 本節有兩種不同風格的VHDL程式碼來描述圖1的系統。
- 第一種風格用 **三態緩衝器** 來表示匯流排，控制電路的輸出邏輯表示式如前述章節所述。
- 第二種程式碼風格用 **多工器** 表示匯流排，用對應到表2的**CASE**敘述描述控制電路的輸出。

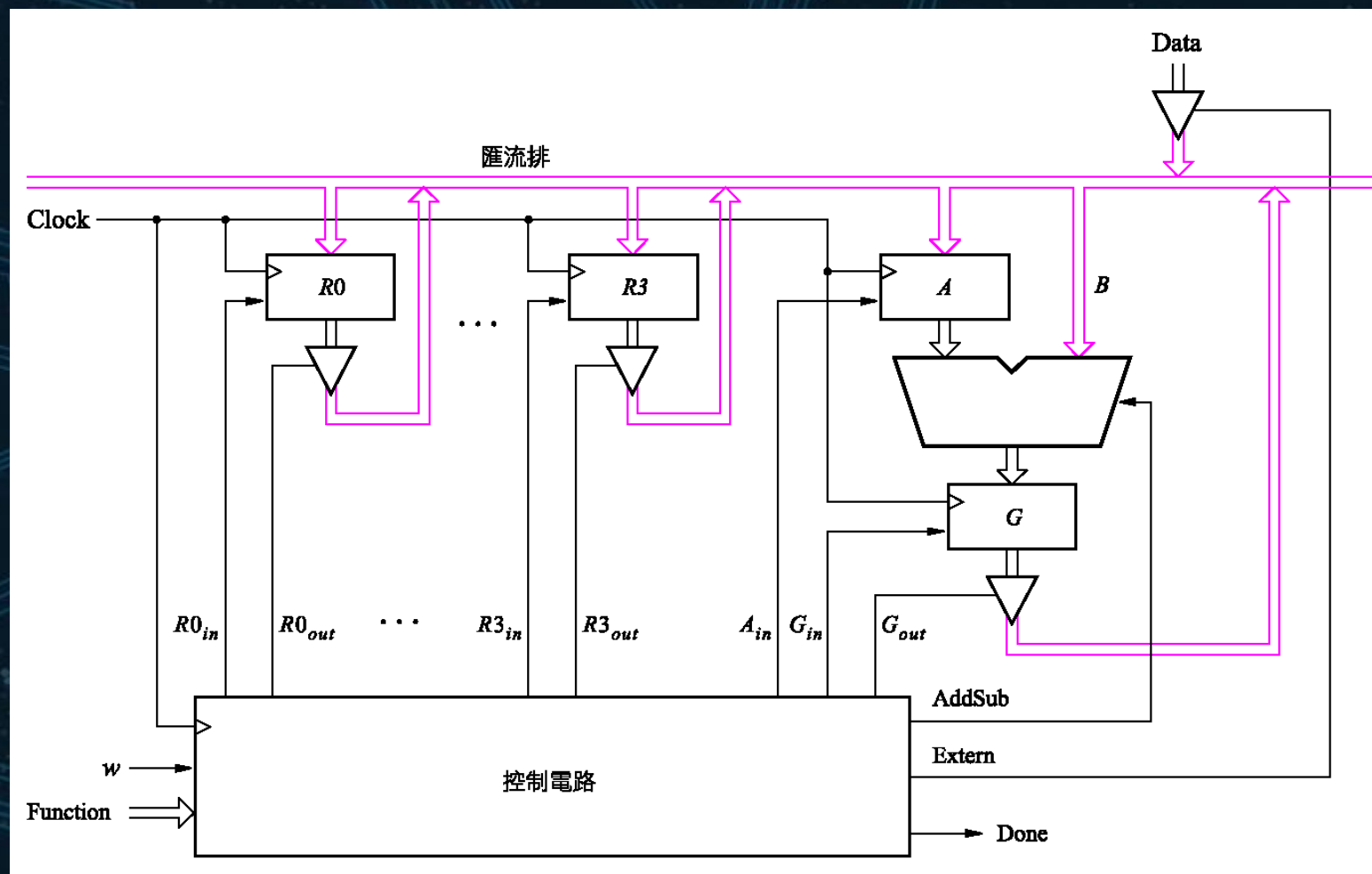


圖4 實作簡單處理器的數位系統



```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
USE ieee.std_logic_unsigned.all ;
ENTITY upcount IS
    PORT (    Clear, Clock    : IN          STD_LOGIC ;
            Q                  : BUFFER     STD_LOGIC_VECTOR(1 DOWNTO 0) ) ;
END upcount ;

ARCHITECTURE Behavior OF upcount IS
BEGIN
    upcount: PROCESS ( Clock )
    BEGIN
        IF (Clock'EVENT AND Clock = '1') THEN
            IF Clear = '1' THEN
                Q <= "00" ;
            ELSE
                Q <= Q + '1' ;
            END IF ;
        END IF;
    END PROCESS;
END Behavior ;
```

圖 5 具有同步重置的二位元上數計數器



使用多工器和case敘述

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
USE ieee.std_logic_signed.all ;
USE work.subccts.all ;

ENTITY proc IS
    PORT (    Data          : IN          STD_LOGIC_VECTOR(7 DOWNT0 0) ;
             Reset, w       : IN          STD_LOGIC ;
             Clock         : IN          STD_LOGIC ;
             F, Rx, Ry      : IN          STD_LOGIC_VECTOR(1 DOWNT0 0) ;
             Done           : BUFFER      STD_LOGIC ;
             BusWires       : INOUT       STD_LOGIC_VECTOR(7 DOWNT0 0) ) ;
END proc ;

ARCHITECTURE Behavior OF proc IS
    SIGNAL Rin, Rout : STD_LOGIC_VECTOR(0 TO 3) ;
    SIGNAL Clear, High, AddSub : STD_LOGIC ;
    SIGNAL Extern, Ain, Gin, Gout, FRin : STD_LOGIC ;
    SIGNAL Count, Zero : STD_LOGIC_VECTOR(1 DOWNT0 0) ;
    SIGNAL T, I, X, Y : STD_LOGIC_VECTOR(0 TO 3) ;
    SIGNAL R0, R1, R2, R3 : STD_LOGIC_VECTOR(7 DOWNT0 0) ;
    SIGNAL A, Sum, G : STD_LOGIC_VECTOR(7 DOWNT0 0) ;
    SIGNAL Func, FuncReg : STD_LOGIC_VECTOR(1 TO 6) ;
    ... con't
```

圖6 處理器的程式碼（a部分）



BEGIN

```
Zero <= "00" ; High <= '1' ;  
Clear <= Reset OR Done OR (NOT w AND T(0)) ;  
counter: upcount PORT MAP (Clear, Clock, Count ) ;  
decT: dec2to4 PORT MAP ( Count, High, T ) ;  
Func <= F & Rx & Ry ;  
FRin <= w AND T(0) ;  
functionreg: regn GENERIC MAP ( N => 6 )  
    PORT MAP ( Func, FRin, Clock, FuncReg ) ;  
decI: dec2to4 PORT MAP ( FuncReg(1 TO 2), High, I ) ;  
decX: dec2to4 PORT MAP ( FuncReg(3 TO 4), High, X ) ;  
decY: dec2to4 PORT MAP ( FuncReg(5 TO 6), High, Y ) ;  
  
Extern <= I(0) AND T(1) ;  
Done <= ((I(0) OR I(1)) AND T(1)) OR ((I(2) OR I(3)) AND T(3)) ;  
Ain <= (I(2) OR I(3)) AND T(1) ;  
Gin <= (I(2) OR I(3)) AND T(2) ;  
Gout <= (I(2) OR I(3)) AND T(3) ;  
AddSub <= I(3) ;  
  
... con't
```

圖7 處理器的程式碼 (b部分)



```
RegCntl:
FOR k IN 0 TO 3 GENERATE
    Rin(k) <= ((I(0) OR I(1)) AND T(1) AND X(k)) OR
              ((I(2) OR I(3)) AND T(3) AND X(k)) ;
    Rout(k) <= (I(1) AND T(1) AND Y(k)) OR
              ((I(2) OR I(3)) AND ((T(1) AND X(k)) OR (T(2) AND Y(k)))) ;
END GENERATE RegCntl ;
tri_extern: trin PORT MAP ( Data, Extern, BusWires ) ;
reg0: regn PORT MAP ( BusWires, Rin(0), Clock, R0 ) ;
reg1: regn PORT MAP ( BusWires, Rin(1), Clock, R1 ) ;
reg2: regn PORT MAP ( BusWires, Rin(2), Clock, R2 ) ;
reg3: regn PORT MAP ( BusWires, Rin(3), Clock, R3 ) ;
tri0: trin PORT MAP ( R0, Rout(0), BusWires ) ;
tri1: trin PORT MAP ( R1, Rout(1), BusWires ) ;
tri2: trin PORT MAP ( R2, Rout(2), BusWires ) ;
tri3: trin PORT MAP ( R3, Rout(3), BusWires ) ;
regA: regn PORT MAP ( BusWires, Ain, Clock, A ) ;
alu:
WITH AddSub SELECT
    Sum <=  A + BusWires WHEN '0',
            A - BusWires WHEN OTHERS ;
regG: regn PORT MAP ( Sum, Gin, Clock, G ) ;
triG: trin PORT MAP ( G, Gout, BusWires ) ;
END Behavior ;
```

圖8 處理器的程式碼（c 部分）



... (ENTITY declaration not shown)

ARCHITECTURE Behavior OF proc IS

SIGNAL X, Y, Rin, Rout : STD_LOGIC_VECTOR(0 TO 3) ;

SIGNAL Clear, High, AddSub : STD_LOGIC ;

SIGNAL Extern, Ain, Gin, Gout, FRin : STD_LOGIC ;

SIGNAL Count, Zero, **T, I : STD_LOGIC_VECTOR(1 DOWNT0 0) ;**

SIGNAL **R0, R1, R2**, R3 : STD_LOGIC_VECTOR(7 DOWNT0 0) ;

SIGNAL A, Sum, G : STD_LOGIC_VECTOR(7 DOWNT0 0) ;

SIGNAL Func, FuncReg, Sel : STD_LOGIC_VECTOR(1 TO 6) ;

BEGIN

Zero <= "00" ; High <= '1' ;

Clear <= Reset OR Done OR (NOT w AND NOT T(1) AND NOT T(0)) ;

counter: **upcount PORT** MAP (Clear, Clock, Count) ;

T <= Count ;

Func <= F & Rx & Ry ;

FRin <= w AND NOT T(1) AND NOT T(0) ;

functionreg: regn **GENERIC** MAP (N => 6)

PORT MAP (Func, FRin, Clock, FuncReg) ;

I <= FuncReg(1 TO 2) ;

decX: **dec2to4 PORT MAP** (FuncReg(3 TO 4), High, X) ;

decY: dec2to4 PORT MAP (FuncReg(5 TO 6), High, Y) ;

controls: PROCESS (T, I, X, Y)

BEGIN

... con't



```
Extern <= '0' ; Done <= '0' ; Ain <= '0' ; Gin <= '0' ;
Gout <= '0' ; AddSub <= '0' ; Rin <= "0000" ; Rout <= "0000" ;
CASE T IS
    WHEN "00" => -- no signals asserted in time step T0
    WHEN "01" => -- define signals asserted in time step T1
        CASE I IS
            WHEN "00" => -- Load
                Extern <= '1' ; Rin <= X ; Done <= '1' ;
            WHEN "01" => -- Move
                Rout <= Y ; Rin <= X ; Done <= '1' ;
            WHEN OTHERS => -- Add, Sub
                Rout <= X ; Ain <= '1' ;
        END CASE ;

    WHEN "10" => -- define signals asserted in time step T2
        CASE I IS
            WHEN "10" => -- Add
                Rout <= Y ; Gin <= '1' ;
            WHEN "11" => -- Sub
                Rout <= Y ; AddSub <= '1' ; Gin <= '1' ;
            WHEN OTHERS => -- Load, Move
        END CASE ;
    WHEN OTHERS => -- define signals asserted in time step T3
        CASE I IS
```

圖10 處理器的另一種程式碼（b 部分）

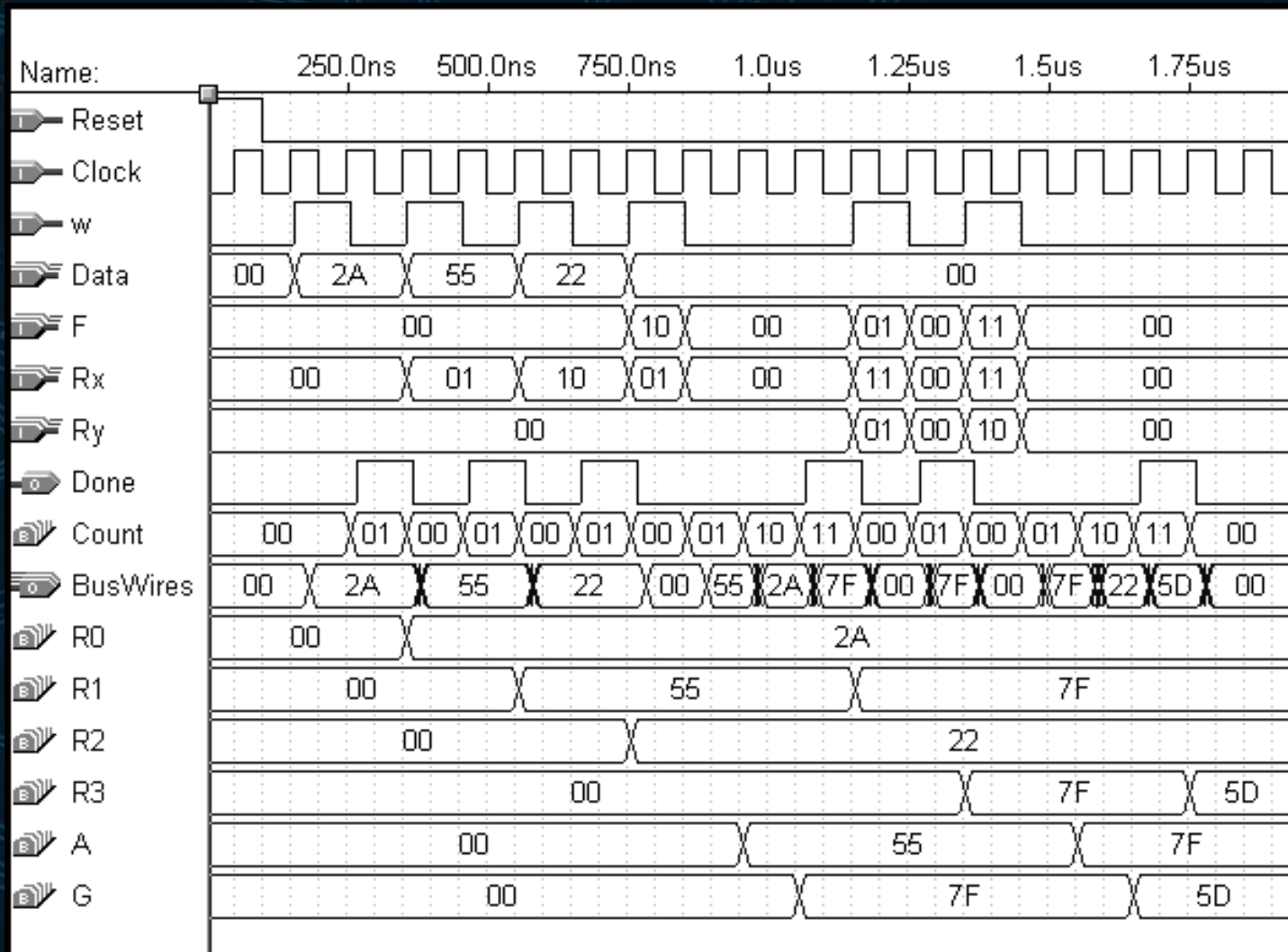


圖11 圖9、圖10中VHDL程式碼的時序模擬